

Retrieval and RAG: Grounding Models in Sources

FRIDAY AI Education — Episode 006 · Printable Companion Paper

Abstract

A large language model, on its own, is a confident stranger. It has read an enormous amount, remembers none of it precisely, and will answer almost any question whether or not it actually knows. That is fine for drafting a tweet and disqualifying for answering "What did our Series A deck promise on gross margin?" This paper is about the technique that closes that gap: **Retrieval-Augmented Generation (RAG)**. RAG keeps the model's fluency but takes away its license to improvise, by fetching real source material at question time and requiring the answer to be built from that material. We cover the mechanism (retrieve, then generate), why citations and **source boundaries** are the point rather than a nicety, why **freshness** is a first-class engineering concern, and why a system that can say "*I don't know*" is more valuable than one that can't. The throughline: memory and retrieval are what turn a generic reasoning engine into something an organization can actually trust with its own facts.

1. Why this matters before we define anything

You are not trying to win a trivia contest against a model. You are trying to build companies in which an AI layer can answer questions about *your* world — your customers, your contracts, your prior decisions — and be right often enough, and honest enough, that serious people will rely on it.

That requirement breaks a plain LLM in two specific ways.

First, a plain model only knows what was baked into its weights during training. It has never seen your cap table, your Notion, or the email you sent a vendor last Tuesday. Ask it about those and it will either refuse or, worse, invent something plausible.

Second, even where the model *has* relevant general knowledge, it can't show you where the answer came from. There is no footnote. For a casual question that's acceptable. For an investor due diligence packet it is a liability, because the value of the answer is inseparable from its provenance.

RAG exists to fix both problems at once. It is, bluntly, the difference between an AI that *sounds* knowledgeable and one that is *accountable*. For an operator, accountability is the whole game. So we will spend most of this paper on the mechanism, and only then connect it to FRIDAY and GBrain — because the mechanism is what you need to be able to explain in a serious room.

2. The core idea in plain language

Here is RAG in one sentence: **before the model answers, go find the relevant documents, and make the model answer using them.**

That's it. There are two moves, and the name tells you their order:

- **Retrieval** — given a question, search a collection of source material and pull back the handful of passages most likely to contain the answer.
- **Generation** — hand those passages to the language model along with the original question, and ask it to write an answer *grounded in what it was just handed*.

The model is no longer answering from memory. It is answering from an open book that you chose and placed in front of it. The retrieval step decides *which pages of the book are open*; the generation step *reads those pages aloud, in context, in the shape the question asked for*.

Two consequences follow immediately, and they are the reason the technique matters:

1. The answer can be **cited**, because you know exactly which passages were in front of the model.
2. The model can answer about things it was never trained on — your private documents — because those documents arrived at question time, not training time.

Everything else in this paper is detail on those two moves and the failure modes around them.

3. The mechanism underneath

It's worth going one level deeper than "search, then answer," because the *how* of retrieval is where the interesting engineering — and the interesting limitations — live.

3.1 The problem retrieval actually solves

Naive search matches words. If your question says "What's our refund policy?" and the document says "Customers may request reimbursement within 30 days," keyword search can miss it entirely — *refund* and *reimbursement* share no letters in common. Human language says the same thing too many ways for literal matching to be reliable.

Modern retrieval sidesteps this by matching **meaning** instead of spelling.

3.2 Embeddings and semantic similarity

Every passage of text — and every question — gets converted into a list of numbers called an **embedding**. You can think of an embedding as coordinates: a point placed in a very high-dimensional space such that texts with similar *meaning* land near each other, regardless of the exact words they use. "Refund policy" and "reimbursement within 30 days" end up close together. "Refund policy" and "office snack budget" end up far apart.

Two cautions, stated plainly because they're commonly fumbled:

- "Near" here means **semantic similarity** / **vector distance** — closeness in meaning-space. It has nothing to do with geographic or physical closeness. The metaphor of *distance* is mathematical.
- An embedding represents **similarity, not truth**. Two passages being close together means they're *about the same thing*, not that either one is *correct*. Retrieval finds relevant text. It does not verify it. Garbage that is on-topic will be retrieved just as eagerly as gold.

3.3 The full loop

Putting it together, a RAG system runs roughly this sequence at question time:

1. **Embed the question** into a vector.
2. **Search** a vector store (a database of pre-embedded document chunks) for the passages whose vectors are closest to the question's vector. This returns the top k most relevant chunks — often 3 to 10.
3. **Assemble a prompt** that contains the original question *plus* those retrieved chunks, usually with an instruction like: *"Answer using only the sources below. Cite them. If they don't contain the answer, say so."*
4. **Generate** the answer with the LLM, working from that assembled prompt.
5. **Attach citations** pointing back to the specific chunks used.

The documents are chopped into chunks and embedded *ahead of time* — this is the "indexing" work you do once when a document enters the system. Retrieval and generation happen *live*, per question. (This live-retrieval design is what Lewis et al. introduced in 2020; see the reading guide.)

The model's job has quietly changed. It is no longer "recall the answer." It is "read these passages and compose a faithful answer to this question, or report that they don't cover it." That is a far more constrained, far more checkable task — and constraint is exactly what you want when facts matter.

4. Source boundaries: the feature that looks like a restriction

The instruction *"answer using only the sources below"* is the single most important sentence in a RAG system, and it deserves its own section because operators consistently underrate it.

A **source boundary** is the explicit rule that the model may answer *from the retrieved material and nothing else*. Outside the boundary, the model is supposed to decline rather than fall back on its general training or its instinct to be helpful.

Why this is the whole point:

- It makes answers **auditable**. If every claim must trace to a retrieved passage, you can check the claim by reading the passage. No boundary, no audit trail.
- It contains **hallucination**. A model asked to answer from sources is much less likely to invent — though, importantly, not incapable of it. It can still misread or over-extend a passage. The boundary reduces fabrication; it does not abolish it.
- It defines **scope** honestly. A company-wiki bot that answers only from the wiki is a known quantity. One that *sometimes* answers from the wiki and *sometimes* from whatever the base model happened to absorb is a quantity you cannot reason about — and therefore cannot deploy with confidence.

The boundary is what converts "a chatbot that read our docs" into "a system whose answers we can stand behind." It feels like a limitation. It is actually the product.

5. Citations: provenance is the deliverable

In a well-built RAG system the citation is not decoration appended after the fact. It is structurally part of the answer, because you know precisely which chunks went into the prompt.

For a founder, citations do three jobs:

- **Verification.** The reader can click through and confirm. The answer becomes a starting point for trust, not a demand for it.
- **Liability containment.** When the answer to "Did we commit to SOC 2 by Q3?" links to the exact contract clause, the answer is defensible. When it doesn't, you're quoting a rumor with extra steps.
- **Debuggability.** When an answer is wrong, citations tell you *where* it went wrong — bad source, or bad reading of a good source. Without them, every error is a mystery.

The discipline to internalize: **an answer without a source is a claim; an answer with a source is evidence.** In any context where the stakes are real — diligence, compliance, a board question — you want evidence, and you want the citation to be load-bearing, not ornamental.

6. Three business examples

The mechanism is general. Here is what it looks like applied.

Public-content knowledge base. Take everything a person or company has put into the world — essays, talks, podcast transcripts, posts — and make that body of work searchable, quotable, and *cited*. Ask "What's their stated position on hiring before product-market fit?" and get an answer stitched from their actual words, each claim linked to the source. This is the most tractable wedge because the corpus is bounded, the sourcing is unambiguous, and being wrong is cheap to catch. It's a clean first place to prove the pattern.

Company wiki Q&A. New hire asks, "What's our policy on contractor equity?" The system retrieves the relevant wiki sections and answers from them, with links — instead of the employee pinging three people or, worse, guessing. The win isn't magic; it's that the answer is grounded in the canonical document and says so, which is exactly what a policy question needs.

Investor due diligence packet. The highest-stakes case, and the clearest illustration of why source boundaries matter. An investor asks pointed questions about revenue recognition, churn definitions, or prior commitments. Every answer must come *only* from the diligence corpus you've assembled, must cite the underlying document, and must say "*that information isn't in the provided materials*" when it genuinely isn't. A model that improvises here doesn't save time — it manufactures legal and reputational risk at machine speed. The boundary and the citation are not features of this product. They *are* the product.

7. Freshness: the answer was true *when*?

A RAG system is only as current as its index. This is the failure mode operators forget about, because it's invisible until it bites.

Picture the diligence bot answering "What is our current monthly burn?" from a board deck that was indexed four months ago. The answer will be confident, well-cited, and **wrong** — because the source is stale, not because the mechanism failed. The retrieval worked perfectly. It retrieved an obsolete truth.

Freshness is the property that the indexed sources reflect current reality, and it's an ongoing operational responsibility, not a one-time setup:

- Documents change; the index must be **re-indexed** when they do, or it will keep confidently serving the old version.
- Some facts are timeless (a definition); some decay fast (a runway figure). The system should know the difference, and ideally surface *when* a source was last updated alongside the answer.
- The most dangerous answer is not the one that's obviously wrong. It's the stale one that's *plausibly* right — well-formatted, properly cited, and quietly out of date.

Freshness is where RAG stops being a clever retrieval trick and becomes a system with operational obligations. Indexing is not a launch task you complete. It is a process you maintain.

8. "I don't know" as a feature, not a failure

The most underrated behavior in a grounded system is the refusal.

A plain LLM is built to be helpful and will answer almost anything — which means it will answer questions it has no business answering, with the same fluent confidence it brings to questions it can actually support. In a grounded system you want the opposite reflex. When the retrieved sources don't contain the answer, the correct output is some honest form of:

"The provided sources don't cover this."

This is **"I don't know" behavior**, and engineering it deliberately is what separates a tool you can trust from one you merely enjoy.

Why it's worth designing for:

- A system that *never* says "I don't know" is not omniscient. It is **untrustworthy** — because it gives you the same confident tone whether or not it's standing on solid ground, and you can't tell the cases apart.
- Calibrated refusal makes the *positive* answers worth more. If the system declines when it lacks support, then when it *does* answer, the answer carries weight. Refusal is what makes confidence meaningful.
- In high-stakes settings — diligence, legal, financial — a clean "not in the sources" is the *right* answer. Filling the gap with a guess isn't helpfulness; it's the most expensive kind of mistake, because it's the kind nobody catches until later.

Teach the system the boundary of its own knowledge and let it admit when it's outside that boundary. An AI that knows what it doesn't know is more useful, and more deployable, than one that always has something to say.

9. Where this sits in FRIDAY and GBrain

Now we can connect the mechanism to the architecture, having earned it.

In FRIDAY's design, the pieces map cleanly:

- **The LLM** is the reasoning and generation engine. It is genuinely capable and genuinely limited — fluent, but with no inherent access to your facts and no native sense of provenance. It is the engine, not the system.
- **GBrain** is the knowledge and memory substrate — the indexed, retrievable body of what FRIDAY knows: documents, prior decisions, captured facts. It is the "open book" that retrieval reads from. **GBrain fact recall** is

RAG pointed inward at your own accumulated knowledge instead of at a public corpus.

- **FRIDAY** is the orchestrator. It runs the retrieve-then-generate loop, enforces source boundaries, attaches citations, watches freshness, and decides when the honest answer is "I don't know."

This is why the slogan is that FRIDAY is an *operator*, not a chatbot. A chatbot generates. An operator retrieves the right context, generates within boundaries, cites its work, and routes anything consequential through approval. RAG is the load-bearing technique that makes GBrain *useful* rather than merely *stored* — memory you can't retrieve against, with sources you can't cite, is just a filing cabinet nobody opens.

A necessary sobering note. Retrieval and generation are the core, but a system organizations actually rely on needs more around them: **permissions** (who may retrieve which sources), **evals** — evaluations, the tests that measure whether answers are grounded and correct — **audit** (a record of what was retrieved and answered), and **human approval gates** before any answer becomes an action. RAG earns the trust. Governance keeps it. Neither substitutes for the other, and an agent is not a magic employee — it's a governed component in a system you remain responsible for.

10. The main limitation to keep in front of you

If you remember one caveat, make it this: **retrieval finds what is relevant, not what is true.**

The system surfaces passages that are *semantically close* to the question. If your corpus contains something wrong, outdated, or contradictory, retrieval will hand it to the model with the same eagerness it hands over the correct passage — and the model will write a fluent, well-cited answer on top of it. Embeddings measure similarity. They do not measure correctness. A confident, sourced, *wrong* answer is entirely possible, and it's the one that will embarrass you, precisely because it looks right.

Which sets the operator's real job. The quality of a RAG system is dominated not by the cleverness of the model but by:

- the **quality** of the source corpus (is it correct?),
- its **freshness** (is it current?),
- the **boundaries** (does the model stay inside the sources?), and
- the **evals** that catch failures before your investor does.

The model is the easy part. The discipline around it is the hard part, and the part that's yours.

11. Compact reference

Concept	One-line definition	Why an operator cares
RAG	Retrieve relevant sources, then generate an answer grounded in them	Turns a generic model into one that answers about <i>your</i> facts, accountably
Retrieval	Searching source material for the passages most relevant to a question	Decides which "open book" pages the model gets to read
Generation	The LLM composing an answer from the retrieved passages	Where fluency lives — but now constrained to the sources

Concept	One-line definition	Why an operator cares
Embedding	A text converted to coordinates in meaning-space	Lets search match meaning, not just keywords; represents similarity, <i>not</i> truth
Semantic similarity / vector distance	Closeness in meaning, measured mathematically	How retrieval ranks passages; nothing to do with physical closeness
Source boundary	The rule that the model answers <i>only</i> from retrieved material	Makes answers auditable and contains hallucination
Citation	A link from each claim back to its source passage	Converts a claim into defensible evidence
Freshness	The index reflecting current reality	A stale-but-plausible answer is the dangerous one
"I don't know" behavior	Declining when the sources don't support an answer	Calibrated refusal is what makes the confident answers worth trusting

12. Vocabulary

- **RAG (Retrieval-Augmented Generation).** A method where a model retrieves relevant documents at question time and generates its answer from them, rather than from training memory alone. (*Say it as "RAG" — retrieval augmented generation. Not "RAIG."*)
- **Retrieval.** The search step: finding the passages most relevant to a question.
- **Generation.** The writing step: the LLM composing an answer from those passages.
- **Embedding.** A numeric representation of text positioned so that similar meanings land near each other. Represents *similarity*, not truth.
- **Vector store.** The database holding pre-embedded document chunks that retrieval searches.
- **Chunk.** A passage-sized piece of a document, embedded and indexed as a retrievable unit.
- **Semantic similarity / vector distance.** Closeness *in meaning*, measured as distance between embeddings. Not geographic.
- **Top-k.** The number of most-relevant chunks retrieval returns per question (often 3–10).
- **Source boundary.** The enforced rule that the model answers only from retrieved sources.
- **Citation.** A pointer from a claim to the specific source it came from.
- **Freshness.** The degree to which indexed sources reflect current reality; maintained by re-indexing.
- **"I don't know" behavior.** Deliberately designed refusal when sources don't support an answer.
- **Evals.** Evaluations — tests that measure whether a system's answers are grounded, correct, and well-behaved. (*Say "evals," not "evils."*)
- **Hallucination.** A fluent, confident answer that isn't supported by any real source. RAG reduces it; it does not eliminate it.

13. Reading guide

Two anchors, read in this order. You do not need to read either cover to cover — read with a purpose.

Pinecone — *What is RAG?* → <https://www.pinecone.io/learn/retrieval-augmented-generation/>

Start here. The clearest plain-language walkthrough of the retrieve-then-generate loop and the role of embeddings and a vector store. *Read for:* a concrete mental model of how a question becomes a search becomes a grounded answer. *Skim past* any vendor-specific product framing — the mechanism is what you're after. ~15 minutes.

Lewis et al. 2020 — *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks* → <https://arxiv.org/abs/2005.11401>

The original paper that named and formalized the approach. *Read for:* the abstract and introduction — enough to understand the founding insight, that combining a retriever with a generator beats a generator alone on tasks that require real-world facts. *Don't get stuck* in the architecture math and benchmark tables; that's not what you need from it. The point you're extracting is conceptual: live retrieval at question time, as a deliberate design, is years old and well-established — not a 2026 novelty. ~20 minutes for the parts that matter.

14. Walking questions

For the next walk. Annotate, don't just nod.

1. Which part of RAG could you explain — clearly, no notes — to a sharp investor right now? Where would you reach for a slide instead of saying it?
2. What part still feels blurry? (Embeddings? Why "similar" isn't "true"? How freshness is maintained?) Name the specific gap.
3. Where does this already live in FRIDAY and GBrain — and where is GBrain still a filing cabinet you can't retrieve against?
4. What is one **bounded, low-stakes** workflow where you could test RAG safely this month — small corpus, clear sources, cheap to be wrong? (The public-content knowledge base is the obvious candidate. What would "good" look like, and how would an eval catch "bad"?)

One-sentence takeaway

RAG makes a language model useful to an organization by forcing it to answer from real, current, cited sources — and to admit when those sources fall silent.